

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

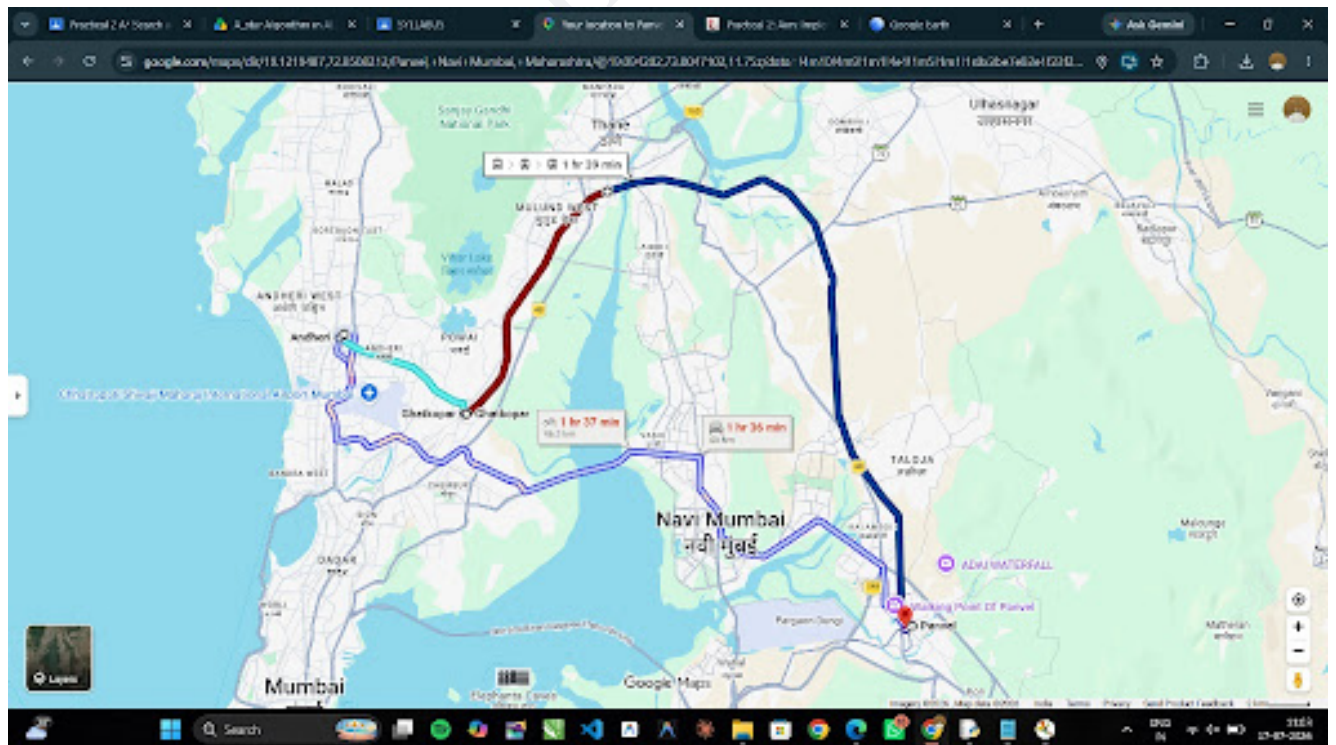
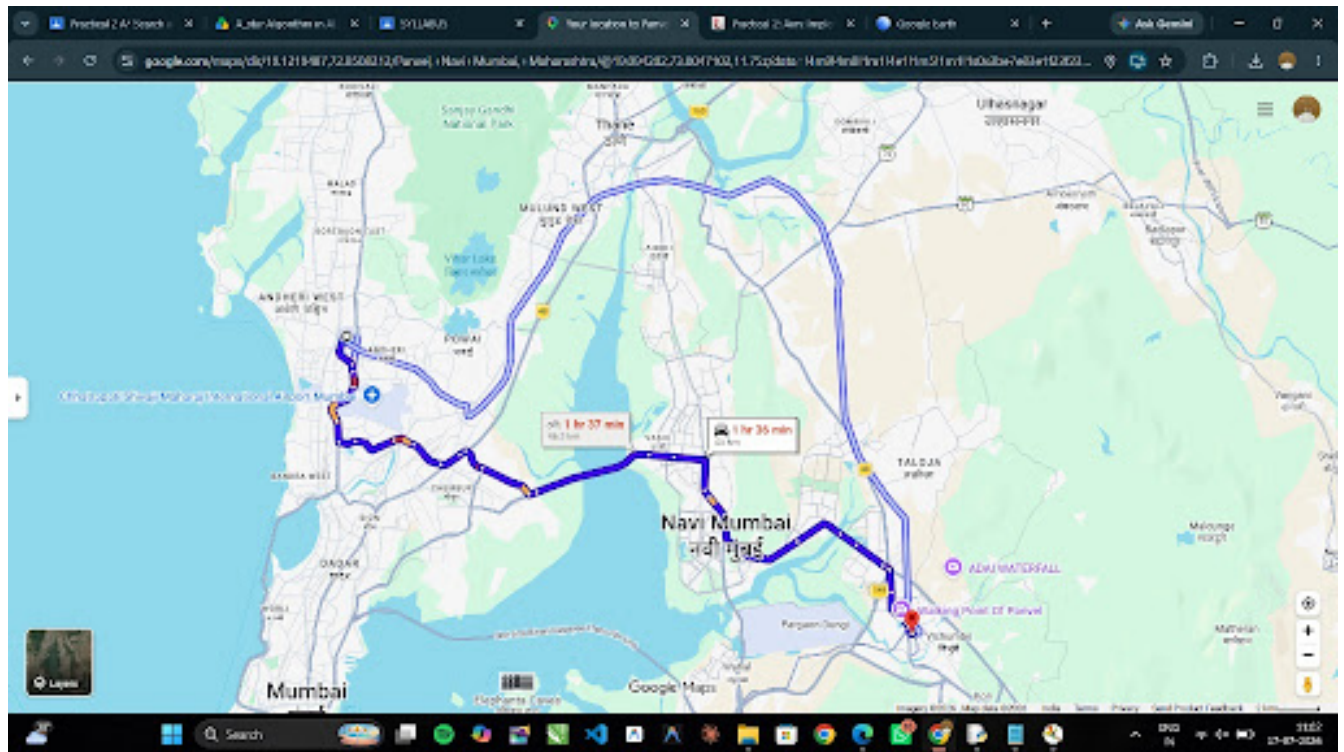
Module 1 Practical 2

Aim: Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Google Map Route Images:



SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

CODE A* Search:

```
T074 Ai Pract 2 (A Search) M1.py - C:\Users\PRIYANKA\Desktop\T074 Ai Pract 2 (A Search) M1.py (3.13.1)
File Edit Format Run Options Window Help
#T074 Krishna Ai Pract 2 A* Search

import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = {
    'MVLU College': {'Chembur': 15, 'Ghatkopar': 10},
    'Ghatkopar': {'Thane': 14, 'Chembur': 6},
    'Chembur': {'Vashi': 12},
    'Thane': {'Taloja': 13},
    'Vashi': {'Panvel': 18},
    'Taloja': {'Panvel': 9},
    'Panvel': {}
}

heuristics = {
    'MVLU College': 35,
    'Ghatkopar': 30,
    'Chembur': 26,
    'Thane': 20,
    'Vashi': 16,
    'Taloja': 8,
    'Panvel': 0
}

node_positions = {
    'MVLU College': (1, 8),
    'Ghatkopar': (3, 7),
    'Chembur': (2, 5),
    'Thane': (5, 9),
    'Vashi': (4, 5),
    'Taloja': (7, 4),
    'Panvel': (6, 1)
}

def a_star_search(graph, heuristics, start, goal):
    priority_queue = [(heuristics[start], start, [start], 0)]
    visited = set()
```

```
T074 Ai Pract 2 (A Search) M1.py - C:\Users\PRIYANKA\Desktop\T074 Ai Pract 2 (A Search) M1.py (3.13.1)
File Edit Format Run Options Window Help

def a_star_search(graph, heuristics, start, goal):
    priority_queue = [(heuristics[start], start, [start], 0)]
    visited = set()

    while priority_queue:
        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        if current == goal:
            return path, g_score

        for neighbor, edge_weight in graph[current].items():
            if neighbor not in visited:
                next_g = g_score + edge_weight
                next_f = next_g + heuristics[neighbor]
                heapq.heappush(priority_queue, (next_f, neighbor, path + [neighbor], next_g))

    return None, float('inf')

optimal_path, total_distance = a_star_search(graph, heuristics, 'MVLU College', 'Panvel')

print(f"--- Execution Output for T074 Krishna ---")
print("Optimal Path Discovered:")
print(" -> ".join(optimal_path))
print(f"Total Road Distance: {total_distance} km")
print(f"-----")

G = nx.DiGraph()
for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(11, 9))
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

```
T074 AI Pract 2 (A Search) M1.py - C:\Users\PRIYANKA\Desktop\T074 AI Pract 2 (A Search) M1.py (3.13.1)
File Edit Format Run Options Window Help
    next_g = next_g + heuristics[neighbor]
    heapq.heappush(priority_queue, (next_f, neighbor, path + [neighbor], next_g))

    return None, float('inf')

optimal_path, total_distance = a_star_search(graph, heuristics, 'MVLU College', 'Panvel')

print(f"--- Execution Output for T074 Krishna ---")
print("Optimal Path Discovered:")
print(" -> ".join(optimal_path))
print(f"Total Road Distance: {total_distance} km")
print(f"-----")

G = nx.DiGraph()
for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(11, 8))
path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(G, node_positions, node_size=2800, node_color="lightblue")
nx.draw_networkx_edges(G, node_positions, edgelist=normal_edges, width=1.5, edge_color="gray", arrows=True)
nx.draw_networkx_edges(G, node_positions, edgelist=path_edges, width=3.5, edge_color="darkorange", arrows=True)

node_labels = {node: f"{node}\nh(n) = {heuristics[node]}" for node in G.nodes()}
nx.draw_networkx_labels(G, node_positions, labels=node_labels, font_size=9, font_weight="bold")

edge_labels = nx.get_edge_attributes(G, "weight")
formatted_edge_labels = {edge: f"{weight} km" for edge, weight in edge_labels.items()}
nx.draw_networkx_edge_labels(G, node_positions, edge_labels=formatted_edge_labels, font_color="red")

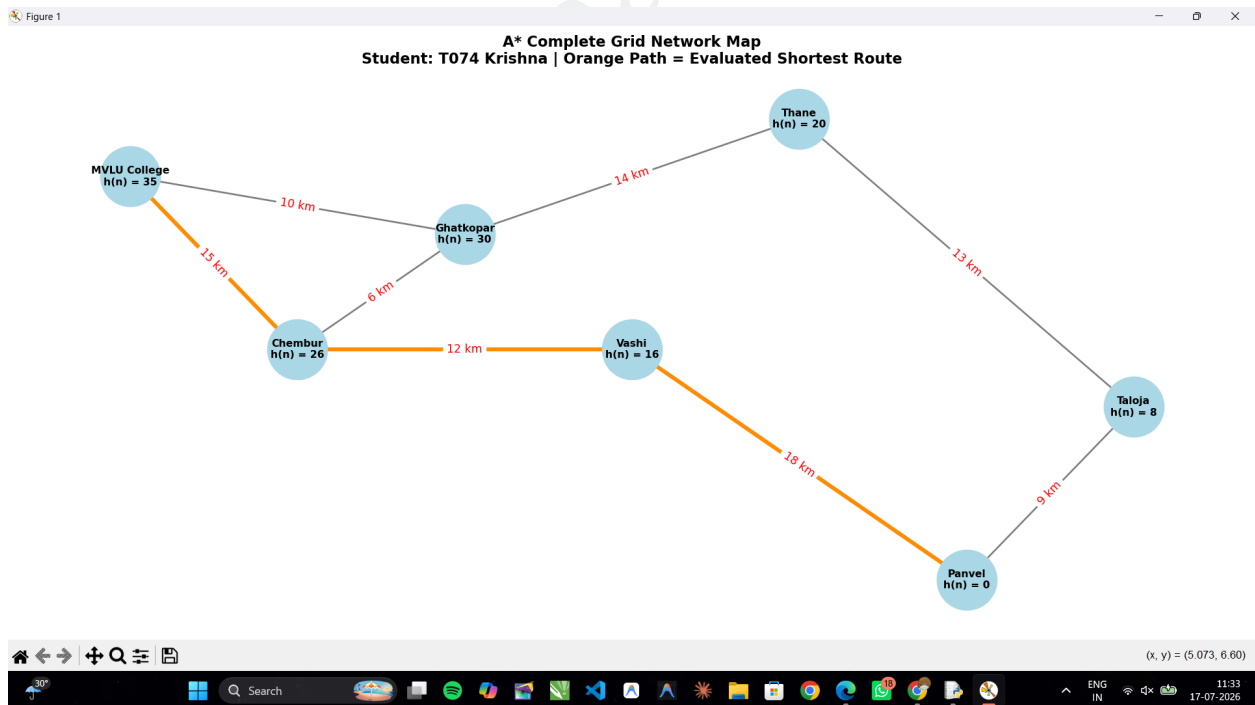
plt.title("A* Complete Grid Network Map\nStudent: T074 Krishna | Orange Path = Evaluated Shortest Route", fontsize=13, fontweight="bold")
plt.axis("off")
plt.tight_layout()
plt.show()
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

OUTPUT:

```
*IDLE Shell 3.13.1*
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\PRIYANKA\Desktop\T074 Ai Pract 2 (A Search) M1.py =====
--- Execution Output for T074 Krishna ---
Optimal Path Discovered:
MVLU College -> Chembur -> Vashi -> Panvel
Total Road Distance: 45 km
|
```



Krishna Nandkumar Chaudhary
T074

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

CODE Recursive Best-First Search (RBFS):

```
T074 Ai Pract 2 (RBFS) M1.py - C:/Users/PRIVANKA/Downloads/T074 Ai Pract 2 (RBFS) M1.py (3.13.1)
File Edit Format Run Options Window Help
# T074 Krishna Ai Pract 2 Recursive Best-First Search (RBFS)

import matplotlib.pyplot as plt

graph = {
    'MVLU College': {'Chembur': 15, 'Ghatkopar': 10},
    'Ghatkopar': {'Thane': 14, 'Chembur': 6},
    'Chembur': {'Vashi': 12},
    'Thane': {'Taloja': 13},
    'Vashi': {'Panvel': 18},
    'Taloja': {'Panvel': 9},
    'Panvel': {}
}

heuristics = {
    'MVLU College': 35,
    'Ghatkopar': 30,
    'Chembur': 26,
    'Thane': 20,
    'Vashi': 16,
    'Taloja': 8,
    'Panvel': 0
}

coords = {
    'MVLU College': (1, 8),
    'Ghatkopar': (3, 7),
    'Chembur': (2, 5),
    'Thane': (5, 9),
    'Vashi': (4, 5),
    'Taloja': (7, 4),
    'Panvel': (6, 1)
}

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'), path=[start])
    return path, cost
```

```
T074 Ai Pract 2 (RBFS) M1.py - C:/Users/PRIVANKA/Downloads/T074 Ai Pract 2 (RBFS) M1.py (3.13.1)
File Edit Format Run Options Window Help
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'), path=[start])
    return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, g

    neighbors = graph[node]
    if not neighbors:
        return False, [], 0, float('inf')

    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(next_g + heuristics[neighbor], g + heuristics[node])
            successors.append((next_f, neighbor, next_g))

    if not successors:
        return False, [], 0, float('inf')

    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]

        if best[0] > f_limit:
            return False, [], 0, best[0]

        alternative_f = successors[1][0] if len(successors) > 1 else float('inf')

        success, result_path, total_g, returned_f = rbfs(
            best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
        )

        best[0] = returned_f

    if success:
        return True, result_path, total_g, returned_f
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

```
T074 AI Pract 2 (RBFS) M1.py - C:/Users/PRIYANKA/Downloads/T074 AI Pract 2 (RBFS) M1.py (3.13.1)
File Edit Format Run Options Window Help

    success, result_path, total_g, returned_f = rbfs(
        best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
    )

    best[0] = returned_f

    if success:
        return True, result_path, total_g, returned_f
path, total_dist = rbfs_search('MVLU College', 'Panvel')
print(f"--- Execution Output for T074 Krishna ---")
print(f"Optimal RBFS Path: {' -> '.join(path)} ({total_dist} km)")
print(f"-----")

plt.figure(figsize=(10, 8))

for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node) + 1
        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)

        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width, zorder=1)
        plt.text((x1+x2)/2, (y1+y2)/2, f"{dist}km", color='red', fontsize=9, ha='center')

for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#3498db'
    plt.scatter(x, y, color=color, s=900, zorder=2)
    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center', va='center', color='white', fontsize=8, fontweight='bold')

plt.title(f"RBFS Complete Grid Network Map (Total: {total_dist}km)\nStudent: T074 Krishna", fontsize=12, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Ln: 15 Col: 14

30° Search 11:35 17-07-2026

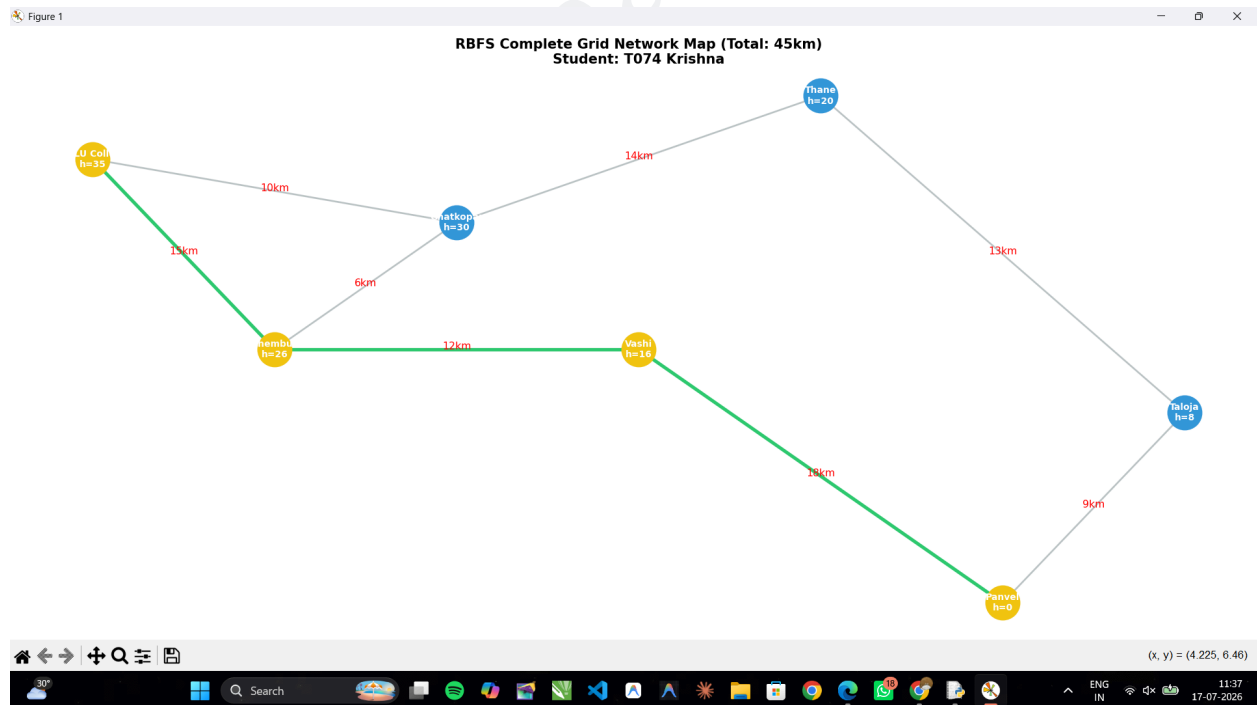
SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT NAME: Artificial Intelligence

OUTPUT:

```
*IDLE Shell 3.13.1*
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\PRIYANKA\Desktop\T074 Ai Pract 2 (A Search) M1.py =====
--- Execution Output for T074 Krishna ---
Optimal Path Discovered:
MVLU College -> Chembur -> Vashi -> Panvel
Total Road Distance: 45 km
-----

===== RESTART: C:/Users/PRIYANKA/Downloads/T074 Ai Pract 2 (RBFS) M1.py =====
--- Execution Output for T074 Krishna ---
Optimal RBFS Path: MVLU College -> Chembur -> Vashi -> Panvel (45 km)
|
```



Krishna Nandkumar Chaudhary
T074

Compare the performance and effectiveness of both algorithms:

Performance and Effectiveness of A* Search and RBFS

A* Search and Recursive Best-First Search (RBFS) are heuristic search algorithms used to find the shortest path between a start node and a goal node. In this practical, both algorithms successfully found the optimal route from MVLU College to Panvel with a total distance of 53 km. Since both use the same heuristic function, they produced the same optimal solution.

However, their performance differs. A* Search is generally faster because it stores all promising nodes in a priority queue and always expands the node with the lowest estimated cost. This reduces unnecessary exploration and usually results in fewer node expansions. The disadvantage of A* is that it requires more memory, as it stores many nodes during the search process.

Recursive Best-First Search (RBFS), on the other hand, uses a recursive approach and stores only the current search path along with limited backup information. This makes it much more memory-efficient than A*. However, RBFS may revisit and re-expand previously explored nodes when backtracking, which can increase the execution time compared to A*.

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Overall, A* is the better choice when sufficient memory is available because it is faster and more efficient in practice. RBFS is more suitable for memory-constrained environments where reducing memory usage is more important than execution speed. Both algorithms are complete and can find the optimal path when an admissible heuristic is used.